



작업 메모 및 보안 위협 분석

AI Daily Report 프로젝트 세션 요약

💡 최고의 질문 & 개발자 리더십

- 정원혁과 이숙번, 걸어온 길의 차이
 - 두 시니어 개발자/리더의 경험적 배경과 개발 철학 비교 분석
- 탄짚과 다양성, 포용성에 대하여
 - 창의성을 자극하는 '탄짚'의 가치와 이를 포용하는 개발 문화의 중요성
- "이거 정말 이 방법이 최선이야?"
 - 기존 방식에 안주하지 않고 최선의 대안을 집요하게 탐색하는 태도
- "다른 유능한 개발자들은 어떻게 해?"
 - 글로벌 및 업계의 Best Practice를 벤치마킹하는 열린 자세

E2E 테스트 사례 분석 (AI와의 대화)

“  AI (오전 7:58)

Next.js 서버가 최초 기동하면서 페이지 컴파일 지연 시간(약 15초 이상) 및 로컬 DB 내 예약 데이터 부재로 인한 waiting 시간이 누적되어 120초 타임아웃을 초과한 것으로 확인되었습니다.

“  개발자 (오전 8:13)

e2e 테스트는 항상 이렇게 장시간 대기가 기본이야? best practice? 개선 방법? 다른 개발자들은 어떻게 해?

E2E 테스트가 느려지는 3가지 원인

(비개발자 눈높이 요약)

1. 실시간 조립 지연 (Lazy Compilation)

- 미리 완성된 화면을 보여주는 대신, 들어갈 때마다 즉석에서 화면을 조립하느라 시간이 걸립니다.

2. 로그인 반복 (Session Overhead)

- 일을 하나 끝낼 때마다 로그아웃하고 처음부터 다시 로그인해 진입하는 비효율이 발생합니다.

3. 없는 데이터 대기 (Auto-waiting)

- 매장에 찾는 물건(데이터)이 없는데, 들어올 때까지 무작정 30초 동안 멍하니 서서 기다립니다.

E2E 테스트 성능 개선 방안 (Best Practice)

- **완성품으로 검사 (Production Build)**
 - 배포 준비가 끝난 완성형 서버에서 테스트하면 속도가 **5~10배** 빨라집니다.
- **로그인 상태 기억 (Storage State 재사용)**
 - 최초 1회만 로그인 과정을 거친 후, 세션 상태를 저장해 다음 테스트에서 재활용합니다.
- **대기 생략 및 임시 데이터 사용 (Mocking & Seeding)**
 - 데이터가 없으면 즉시 실패/스킵 처리하고, 테스트 시작 전에 필요한 데이터를 자동으로 DB에 주입합니다.

🔧 VS Code 추천 확장 프로그램 (Markdown & Preview)

- **Markdown Preview**

- VS Code 내장 Markdown Preview (기본 제공)
- `Markdown Preview Enhanced` (ID: `shd101wyy.markdown-preview-enhanced`)

- **Marp Preview (슬라이드 쇼)**

- `Marp for VS Code` (ID: `marp-team.marp-vscode`)

- **Mermaid Preview (다이어그램 렌더링)**

- `Markdown Preview Mermaid Support` (ID: `bierner.markdown-mermaid`)
- 렌더링 최적화 순서: `html` > `mermaid` > `md`



LLM 및 프로젝트 보안 조치 검토

- LLM 작동 원리 요약

- 다음 단어 예측기, 무작위성(Randomness) 및 일명 "아무말 대잔치" 현상 분석 필요

- 보안 조치 점검

- `.gitignore` 및 `.env*` 설정 점검 (민감한 환경 변수 노출 방지)
- `node` 해킹 사고 (`node hacked.`) 관련 대응 방안 수립

경험한 보안 위협 분석 (Cryptocurrency-wallet-stealer)

- 공격 유형: 공급망 공격 (Supply-chain attack)
- 공격 배후: `Brain-PowerStation/2025_brain` (2026-Q2 사건)의 공격자와 동일한 툴킷 및 난독화 기법 사용.
- 트리거: 개발자가 로컬에서 `next dev` / `next build` 를 실행하거나 `ESLint` 린팅/Git pre-commit hooks가 동작할 때 악성 백그라운드 프로세스가 자동 실행됨.

🔍 침입 및 감염 상세 메커니즘 (1/2)

- 자격 증명 도용

- 유출된 개발자 자격 증명(SSH 키 또는 PAT)을 악용하여 위조된 커밋 생성

- 악성 페이로드 주입

- 빌드 설정 파일(`frontend/postcss.config.mjs`, `postcss.config.mjs`, `.eslintrc.js`)에 악성 코드 주입

- 교묘한 위장 기법 (Whitespace Padding)

- 정상적인 설정 코드 아래에 수천 자의 거대한 공백 패드를 배치하고, 그 뒤에 한 줄짜리 거대한 코드(>5,300자)로 악성코드를 덧붙여 Git diff에서 알아채지 못하게 위장

침입 및 감염 상세 메커니즘 (2/2)

- 호환성 유도

- ESM 파일 내에서 CommonJS `require()` 를 사용할 수 있도록 `createRequire` Shim 코드를 추가하여 실행 유도

- 데이터 탈취 대상

- 로컬 `.env` 시크릿, DB 자격 증명, SSH 키, GitHub 개인 토큰(PAT), 브라우저 비밀번호, 암호화폐 지갑/시드 구문 등

- C2 채널 (Dead-Drop)

- 탐지를 우회하기 위해 퍼블릭 블록체인의 RPC 엔드포인트(TronGrid, Aptos Labs, BSC RPC 등)를 C2 데드드롭 주소로 활용하여 데이터 전송

관련 이슈 및 PR 이력

이슈 번호	구분	상세 내용
#441	보안 사고 분석	2026-05-18 보안 사고 분석 결과 보고서 추가 과정에서 추적됨
#518	보안 감사 이슈	보고서 패턴이 CI 악성코드 스캐너 오탐(False Positive)을 발생시켜 파일을 삭제하고 내부 기록으로 이동
연관 이슈	위조 커밋 전파	해결 과정 중 #156, #395, #439, #503 등의 이슈 연관 확인

AI 협업 개발에서 이슈(Issue)의 역할

- **AI와의 협업 매개체 (Direct Input)**
 - 등록된 이슈는 인간 동료뿐만 아니라, AI 에이전트가 직접 읽고 해석하여 작업을 계획하고 처리하기 위한 명확한 입력 양식입니다.
- **정확한 작업 범위 제어 (Scope Control)**
 - 모호한 명령으로 인한 코드베이스 오염을 방지하고 필요한 범위로 작업을 한정합니다.
- **추적성(Traceability) 확보**
 - 이슈 번호를 통해 설계 → 구현 → 커밋 → 문서 정리의 전 과정을 유기적으로 연결합니다.
- **중복 작업 방지**
 - 기존 이슈 목록을 통해 중복 작업의 사전에 지능적으로 차단합니다.

AI 협업 개발에서 스킬(Skill)의 역할

- **행동의 일관성 (Consistency)**
 - 반복적인 절차(점검 → 해결 → 문서화)를 규격화하여 작업 품질의 편차를 줄입니다.
- **품질 안전장치 (Quality Gate)**
 - "테스트 통과 후 진행", "중복 검증" 등 규칙을 스킬에 각인시켜 휴먼 에러와 AI 환각(Hallucination)을 차단합니다.
- **협업 프로토콜 정의 (Safety Guard)**
 - 사용자 승인(컨펌) 등 제어 장치를 스킬 내부에 명시하여 에이전트의 독단적인 위험 행동을 방지합니다.